

COMPUTER SCIENCE TRIPOS Part IB – 2021 – Paper 5

8 Concurrent and Distributed Systems (mk428)

You are developing a distributed system in which you want some task to be assigned to exactly one node at any given time. If that node crashes, the task must be automatically reassigned to a different node, but you also want to ensure that there are never two or more nodes executing the task at the same time.

This is known as a *lease*. A lease is a concurrency primitive similar to a lock (only one node may hold a lease at any time); the difference is that a lease times out if it is not renewed for some time. After timing out, another node can acquire the lease.

leader election,
Raft

- (a) Briefly summarise how leader election works in the Raft consensus algorithm, and discuss the commonalities and differences between leader election and a lease. (Focus only on leader election, and ignore the rest of the Raft algorithm. Include the role of the term number in your explanation.) [5 marks]

Answer: Leader election in Raft ensures that there is never more than one leader for a given term number, and term numbers generally increase over time. It works as follows:

- A node starts in the *follower* state. If it doesn't hear from a leader within some timeout, it increments its term number, transitions to *candidate* state, and requests votes from the other nodes for that term.
- A node votes for another node's candidacy if it has not already voted for another node in the same term or a later term. A node that votes for another node transitions to *follower*, if it is not already in that state.
- If a candidate receives positive votes from a quorum of nodes (typically a majority, including itself), it becomes the leader in the term for which it held the election.

This is similar to a lease in that there is only one leader per term, and a new leader is elected if the old leader is inactive for some time. The difference is that it is possible for more than one node to simultaneously be a leader for different terms.

system models,
faults

- (b) In a partially-synchronous system with crash-recovery failures, is it possible to guarantee that a lease is always held by exactly one node? Justify your answer. [5 marks]

Answer: No. Firstly, there may be periods of time during which the lease is not held by any node, because if the leaseholder crashes, it takes some time for the lease to be reassigned to another node. This is because we have to wait for the lease timeout to elapse, and because it takes time to detect the failure of the leaseholder (since failure detection is itself typically based on timeouts).

Secondly, there may be periods of time during which more than one node believe that they hold the lease. This could happen in the following scenario: the current leaseholder pauses execution for an extended period of time (which is allowed by the partially synchronous model); the lease timeout elapses and another node acquires the lease; then the previous leaseholder resumes execution without noticing that it was paused. Now both nodes believe they hold the lease (a form of *split-brain*).

leader election,
quorums

- (c) You are asked to design a lease algorithm for a system in which the set of nodes is

not known in advance, and may change over time. Can the Raft leader election algorithm be used here? Why/why not? [2 marks]

Answer: No. Raft's leader election relies on receiving a quorum of votes, and for a quorum to be defined you need to know the identities of all of the nodes that may vote. Moreover, only one of those known nodes can become the leader. With an unknown set of nodes, neither of these conditions is satisfied.

physical clocks,
clock drift,
quorums, system
models

(d) A colleague proposes the following lease algorithm:

- There are three servers, each storing a value that is initially **null**.
- Assume every client has a unique ID *clientId* $\neq$ **null**. Every 10 seconds, each client that wants to acquire the lease, or currently holds the lease, sends a request (**acquire**, *clientId*) to all three servers.
- When a server with current stored value *v* receives (**acquire**, *n*):
If *v* = **null** $\vee$ *v* = *n*, or if its value was last set more than 30 seconds ago, then it sets its value to *n* and replies **true**. This counts as “setting the value”, even if the value does not change.
If its value was last set to *v* $\neq$ *n* less than 30 seconds ago, it leaves its value unchanged and replies **false**.
- If a client receives two or more **true** responses from the servers, it now holds the lease, otherwise it does not hold the lease.

Discuss the strengths and weaknesses of this algorithm. What faults does it tolerate? What assumptions does it make for its correctness? How might the algorithm be improved to avoid some assumptions or weaknesses? [8 marks]

Answer: Advantages:

- The algorithm tolerates one of the three servers being faulty or unreachable, since it requires only two responses. It also tolerates some lost messages, although too many lost messages will result in a client losing the lease or failing to acquire it in the first place.
- It works with any number of clients; the set of all clients can change over time, and need not be known.
- Under the assumptions below, it ensures that no more than one client acquires the lease within a given 30-second window, since it is not possible for two different clients to both receive two or more **true** responses within less than this time.
- If the leaseholder crashes, it stops making requests, so the 30 seconds since the last update will eventually elapse. Assuming the other clients are constantly trying to acquire the lease, one of them will then be able to take over the lease.
- The algorithm does not assume synchronised clocks (since clients and servers only measure elapsed time, and they do not compare timestamps across clients). If NTP is misconfigured, for example, this algorithm is not affected.

Assumptions:

- It assumes the network delay is less than the 10 second polling interval, otherwise responses may be attributed to the wrong requests. This assumption can be avoided by unambiguously associating responses with requests, e.g. by tagging each request/response pair with a unique ID, or by performing each request/response over a new TCP connection, which effectively does the same thing. (If a TCP connection is reused for multiple requests, per-request tags are still needed: say a client sends two requests over the same TCP connection, and then receives one response. Without uniquely tagging each request/response, the client does not know which of the two requests the response is for.) Clients must ignore responses that arrive after the client has sent its next round of requests (i.e. after 10 seconds), otherwise delayed messages could result in two clients both thinking they have the lease.
- It assumes that measurement of elapsed time is reasonably accurate on each server and client. This can be achieved by using a monotonic clock (not a time-of-day clock that may jump forwards or backwards) and by checking that the rate of clock drift is within reasonable bounds. Google's Spanner is another example of a system that assumes bounded clock drift, so there is precedent for this assumption.
- It assumes that clients do not pause execution for longer than about 20 seconds (timeout minus lease renewal period), since otherwise there could be multiple clients concurrently believing to hold the lease, as discussed in part (b). Strictly speaking, this means that the algorithm must assume the system model is synchronous, since partially-synchronous systems can have pauses of unbounded length.
- It assumes that if a server crashes and restarts, it remembers the value it had stored before the crash. Thus, it needs to be written to durable storage. Alternatively, a server that crashes can wait for 30 seconds before restarting; this would prevent a crashed server from replying `true` to two different clients within a 30-second period, even if the server's value is lost on crash.

Disadvantages:

- The system needs to monitor servers to check for abnormally high rate of clock drift.
- The algorithm does not define a way of explicitly releasing the lease when a client no longer needs it, so the only option is to wait for it to time out. This feature could be added by allowing clients to request servers to set their value back to null, provided that the value still equals the sender's *clientId*.
- It is possible for the system to become livelocked. For example, server 1 first receives the request from client *A* and sets its value to *A*, server 2 similarly sets its value to *B*, and server 3 sets its value to *C*. On every retry, the same value remains set on each server, and no client ever obtains a quorum. One way of addressing this would be for clients to explicitly set server values back to null if they are unable to form a quorum, thus improving another client's chances of obtaining a quorum. Also, the retry timeout could be randomised to reduce the probability of repeated concurrent requests.
- This algorithm has no equivalent to the term number in Raft's leader election (i.e. a value that uniquely identifies a particular lease acquisition event). Therefore, if there are two clients that both believe they are holding the lease, and they communicate with another node, this node has no way of telling which of the two was granted the lease more recently. Adding an incrementing number, similar to Raft's term number, would address this issue.

Not all the above points need to be stated in order to get full marks. Marks are awarded for demonstrating a clear understanding of the issues surrounding distributed systems.
